Streams and Buffers

Section: 5 — File System Operations and Streams

Subtopic: 02 — Streams and Buffers

Estimated Duration: 3–4 hours

Theory Practical

Topics covered: Readable streams, Writable streams, Duplex and Transform streams, piping and chaining, Buffers, encoding

Learning Objectives — This Subtopic

- Explain what streams are and why they are essential for processing large data sets in Node.js
- Create and consume Readable and Writable streams using the `fs` module
- Chain streams together using `pipe()` and the `pipeline()` utility
- Build custom Transform streams to modify data as it flows through
- Work with Buffers to handle raw binary data and convert between encodings
- Apply streams to real-world file processing tasks (CSV/log transformation)

1. Why Streams?

In Section 5-1, you used `fs.readFile()` to load entire files into memory. This works perfectly for small files — a configuration file, a short text document. But consider a 2 GB log file. Calling `readFile()` on it would attempt to load all 2 GB into memory at once, which could crash your process or at minimum cause severe performance degradation.

Streams solve this problem by processing data in small chunks. Instead of loading everything into memory, a stream reads (or writes) a piece at a time, processes it, and moves on. This keeps memory usage constant regardless of the total data size.

> **Analogy:** Think of streams like a conveyor belt in a factory. A conveyor belt does not dump all items onto a table at once — it moves items one at a time past different stations. Each station processes its item and passes it along. The factory does not need a warehouse-sized table; it just needs enough space for one item at each station. Streams work the same way: data flows through your programme in manageable pieces.

1.1 The Memory Problem — A Demonstration

Let us prove the problem with a concrete example. First, create a project folder for this section:

```
mkdir streams-demo && cd streams-demo
```

We will start by generating a large test file, then compare the memory usage of `readFile()` versus a stream.

generate-large-file.js

```
const fs = require('fs');

const filePath = 'large-file.txt';
const writeStream = fs.createWriteStream(filePath);

// Write 1 million lines
for (let i = 0; i < 1000000; i++) {
    writeStream.write(`Line ${i + 1}: This is sample data for demonstrating streams in
Node.js\n`);
}

writeStream.end(() => {
    const stats = fs.statSync(filePath);
    const sizeMB = (stats.size / 1024 / 1024).toFixed(2);
    console.log(`File created: ${filePath} (${sizeMB} MB)`);
});
```

Run it:

```
node generate-large-file.js
```

Output:

```
File created: large-file.txt (73.43 MB)
```

Now compare two approaches to reading this file — one using `readFile()` and one using a stream:

read-entire-file.js

```
const fs = require('fs');

const before = process.memoryUsage().heapUsed;

fs.readFile('large-file.txt', 'utf8', (err, data) => {
    if (err) throw err;

    const after = process.memoryUsage().heapUsed;
    const usedMB = ((after - before) / 1024 / 1024).toFixed(2);

    console.log(`Lines read: ${data.split('\n').length}`);
    console.log(`Memory used: ${usedMB} MB`);
});
```

```
node read-entire-file.js
```

Output:

```
Lines read: 1000001
Memory used: 148.67 MB
```

read-with-stream.js

```
const fs = require('fs');

let lineCount = 0;
const before = process.memoryUsage().heapUsed;

const readStream = fs.createReadStream('large-file.txt', { encoding: 'utf8' });

readStream.on('data', (chunk) => {
    // Count newlines in this chunk
```

```
    const lines = chunk.split('\n').length - 1;
    lineCount += lines;
});

readStream.on('end', () => {
    const after = process.memoryUsage().heapUsed;
    const usedMB = ((after - before) / 1024 / 1024).toFixed(2);

    console.log(`Lines read: ${lineCount}`);
    console.log(`Memory used: ${usedMB} MB`);
});

readStream.on('error', (err) => {
    console.error('Stream error:', err.message);
});
```

`node read-with-stream.js`

Output:

```
Lines read: 1000000
Memory used: 3.21 MB
```

What is happening? The `readFile()` approach loaded the entire 73 MB file into a single string, which consumed roughly 149 MB of heap memory (strings in V8 take approximately double the raw bytes). The stream approach processed the same file in chunks of ~64 KB each, keeping memory usage under 4 MB regardless of the file size. This is the core advantage of streams.

> **Key Insight:** Streams are not just about large files. They are a fundamental abstraction in Node.js. HTTP requests and responses, TCP sockets, compression, encryption — all of these are implemented as streams. Understanding streams unlocks a deeper understanding of how Node.js handles I/O.

---

2. The Four Stream Types

Node.js defines four categories of stream, all found in the `stream` core module:

Stream Type	Description	Example
Readable	A source you can read data *from*	`fs.createReadStream()`, `process.stdin`
Writable	A destination you can write data *to*	`fs.createWriteStream()`, `process.stdout`
Duplex	Both readable and writable (independent channels)	TCP sockets
Transform	A Duplex stream that modifies data as it passes through	`zlib.createGzip()`

This section focuses on Readable, Writable, and Transform streams in depth — these are the ones you will use most frequently. Duplex streams are covered at a conceptual level as they share behaviour with the other types.

---

3. Readable Streams

A Readable stream is any source of data that emits chunks over time. The most common one is `fs.createReadStream()`, which you have already seen. Let us examine it more closely.

3.1 Creating a Readable Stream

`readable-basics.js`

```
const fs = require('fs');

const readStream = fs.createReadStream('large-file.txt', {
    encoding: 'utf8',    // Decode bytes to string (default is Buffer)
    highWaterMark: 1024  // Chunk size in bytes (default is 64 KB)
});

let chunkCount = 0;

readStream.on('data', (chunk) => {
    chunkCount++;
```

```
    if (chunkCount <= 3) {
        console.log(`--- Chunk ${chunkCount} (${chunk.length} characters) ---`);
        console.log(chunk.substring(0, 80) + '...');
    }
});

readStream.on('end', () => {
    console.log(`\nTotal chunks received: ${chunkCount}`);
});

readStream.on('error', (err) => {
    console.error('Error:', err.message);
});
```

```
node readable-basics.js
```

Output:

```
--- Chunk 1 (1024 characters) ---
Line 1: This is sample data for demonstrating streams in Node.js
Line 2: This is sam...
--- Chunk 2 (1024 characters) ---
ple data for demonstrating streams in Node.js
Line 16: This is sample data for demo...
--- Chunk 3 (1024 characters) ---
onstrating streams in Node.js
Line 30: This is sample data for demonstrating stre...

Total chunks received: 75029
```

What is happening? The `highWaterMark` option controls the size of each chunk. We set it to 1024 bytes (1 KB) to illustrate that the file is delivered in many small pieces. The default is 65536 bytes (64 KB). Each time a chunk is ready, the `'data'` event fires. When the entire file has been read, the `'end'` event fires.

3.2 Readable Stream Events

Readable streams emit several events:

Event	When It Fires
`data`	A chunk of data is available to consume
`end`	

	No more data will be emitted — the source is exhausted
`error`	An error occurred (e.g., file not found)
`close`	The underlying resource (e.g., file descriptor) has been closed

> **Important:** Always attach an `'error'` event handler to streams. An unhandled stream error will crash your Node.js process with an uncaught exception.

3.3 Pausing and Resuming

Readable streams operate in one of two modes: **flowing** and **paused**. When you attach a `'data'` listener, the stream enters flowing mode and pushes chunks automatically. You can pause and resume this flow:

pause-resume.js

```
const fs = require('fs');

const readStream = fs.createReadStream('large-file.txt', {
    encoding: 'utf8',
    highWaterMark: 64 * 1024  // 64 KB chunks (default)
});

let chunkCount = 0;

readStream.on('data', (chunk) => {
    chunkCount++;
    console.log(`Chunk ${chunkCount} received (${chunk.length} chars)`);

    if (chunkCount === 3) {
        console.log('Pausing stream for 2 seconds...');
        readStream.pause();

        setTimeout(() => {
            console.log('Resuming stream...');
            readStream.resume();
        }, 2000);
    }
});
```

```
readStream.on('end', () => {
    console.log(`Done. Total chunks: ${chunkCount}`);
});
```

`node pause-resume.js`

> Output:
>
> ```
> Chunk 1 received (65536 chars)
> Chunk 2 received (65536 chars)
> Chunk 3 received (65536 chars)
> Pausing stream for 2 seconds...
> Resuming stream...
> Chunk 4 received (65536 chars)
> Chunk 5 received (65536 chars)
> ...
> Done. Total chunks: 1173
> ```

What is happening? After the third chunk, we call `readStream.pause()`. The stream stops emitting `'data'` events. After 2 seconds, `resume()` restarts the flow. This mechanism is the basis of **backpressure** — a way for a slow consumer to tell a fast producer to wait.

---

4. Writable Streams

A Writable stream is a destination for data. The most common example is `fs.createWriteStream()`.

4.1 Creating a Writable Stream

`writable-basics.js`

```
const fs = require('fs');

const writeStream = fs.createWriteStream('output.txt');

writeStream.write('First line of output\n');
writeStream.write('Second line of output\n');
writeStream.write('Third line of output\n');

// Signal that no more data will be written
```

```
writeStream.end('Final line — stream closed after this\n');

writeStream.on('finish', () => {
    console.log('All data has been written to output.txt');

    // Verify by reading the file
    const content = fs.readFileSync('output.txt', 'utf8');
    console.log('\nFile contents:');
    console.log(content);
});

writeStream.on('error', (err) => {
    console.error('Write error:', err.message);
});
```

```
node writable-basics.js
```

Output:

```
All data has been written to output.txt

File contents:
First line of output
Second line of output
Third line of output
Final line — stream closed after this
```

What is happening? Each `write()` call sends a chunk of data to the file. The `end()` method writes a final piece of data (optional) and then signals that the stream is complete. The `'finish'` event fires once all data has been flushed to the underlying resource.

4.2 Handling Backpressure

The `write()` method returns a boolean. If it returns `false`, the internal buffer is full and you should wait for the `'drain'` event before writing more data. Ignoring this leads to excessive memory usage — exactly the problem streams are meant to solve.

`backpressure-demo.js`

```
const fs = require('fs');

const writeStream = fs.createWriteStream('backpressure-test.txt', {
```

```
    highWaterMark: 1024  // Small buffer for demonstration (1 KB)
});

let i = 0;
const totalWrites = 10000;

function writeData() {
    let ok = true;

    while (i < totalWrites && ok) {
        i++;
        const data = `Entry ${i}: Some data to write to the file\n`;

        if (i === totalWrites) {
            // Last write — use end()
            writeStream.end(data);
        } else {
            // write() returns false when the buffer is full
            ok = writeStream.write(data);
        }
    }

    if (i < totalWrites) {
        // Buffer was full — wait for drain before continuing
        console.log(`Buffer full at entry ${i}. Waiting for drain...`);
        writeStream.once('drain', writeData);
    }
}

writeStream.on('finish', () => {
    console.log(`Done. Wrote ${totalWrites} entries.`);
});

writeData();
```

```
node backpressure-demo.js
```

Output:

```
Buffer full at entry 24. Waiting for drain...
Buffer full at entry 48. Waiting for drain...
Buffer full at entry 72. Waiting for drain...
...
Done. Wrote 10000 entries.
```

What is happening? With a `highWaterMark` of 1 KB, the internal buffer fills up quickly. When `write()` returns `false`, our function stops writing and waits for the `'drain'` event, which fires once the buffer has been flushed to disk. This pattern keeps memory usage stable.

> **Key Insight:** You rarely need to handle backpressure manually. The `pipe()` method (covered next) handles it automatically. But understanding *why* backpressure exists is important — it explains the behaviour of `pipe()` and why streams are memory-efficient.

---

5. Piping Streams

Piping connects a Readable stream to a Writable stream. Data flows from the source into the destination automatically, with backpressure handled for you.

5.1 Basic Pipe

`pipe-basic.js`

```
const fs = require('fs');

const readStream = fs.createReadStream('large-file.txt');
const writeStream = fs.createWriteStream('large-file-copy.txt');

readStream.pipe(writeStream);

writeStream.on('finish', () => {
    const original = fs.statSync('large-file.txt').size;
    const copy = fs.statSync('large-file-copy.txt').size;

    console.log(`Original: ${(original / 1024 / 1024).toFixed(2)} MB`);
    console.log(`Copy:     ${(copy / 1024 / 1024).toFixed(2)} MB`);
    console.log(`Match:    ${original === copy}`);
});

readStream.on('error', (err) => console.error('Read error:', err.message));
writeStream.on('error', (err) => console.error('Write error:', err.message));
```

```
node pipe-basic.js
```

Output:

```
Original: 73.43 MB
Copy:     73.43 MB
Match:    true
```

What is happening? A single line — `readStream.pipe(writeStream)` — copies the entire file. Internally, `pipe()` listens for `'data'` events on the readable, calls `write()` on the writable, and handles backpressure by pausing the readable when the writable's buffer is full.

5.2 Chaining Pipes

`pipe()` returns the destination stream, which allows chaining when the destination is also readable (i.e., a Transform or Duplex stream). A common use case is compressing a file:

pipe-chain.js

```
const fs = require('fs');
const zlib = require('zlib');

const source = fs.createReadStream('large-file.txt');
const gzip = zlib.createGzip();
const destination = fs.createWriteStream('large-file.txt.gz');

source
    .pipe(gzip)         // Compress the data
    .pipe(destination); // Write compressed data to file

destination.on('finish', () => {
    const originalSize = fs.statSync('large-file.txt').size;
    const compressedSize = fs.statSync('large-file.txt.gz').size;
    const ratio = ((1 - compressedSize / originalSize) * 100).toFixed(1);

    console.log(`Original:   ${(originalSize / 1024 / 1024).toFixed(2)} MB`);
    console.log(`Compressed: ${(compressedSize / 1024 / 1024).toFixed(2)} MB`);
    console.log(`Reduction:  ${ratio}%`);
});

source.on('error', (err) => console.error('Source error:', err.message));
destination.on('error', (err) => console.error('Destination error:', err.message));
```

```
node pipe-chain.js
```

Output:

```
Original:   73.43 MB
Compressed: 1.18 MB
Reduction:  98.4%
```

What is happening? Data flows through a pipeline: file → gzip compressor → compressed file. The `zlib.createGzip()` function returns a Transform stream — it reads uncompressed data, compresses it, and emits the compressed data. Because it is both readable and writable, it can sit in the middle of a pipe chain.

5.3 The pipeline() Utility

There is a problem with `pipe()`: if an error occurs in any stream in the chain, the other streams are not automatically closed. This can lead to memory leaks. Node.js provides `stream.pipeline()` to handle this properly:

`pipeline-demo.js`

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream');

pipeline(
    fs.createReadStream('large-file.txt'),
    zlib.createGzip(),
    fs.createWriteStream('large-file-pipeline.txt.gz'),
    (err) => {
        if (err) {
            console.error('Pipeline failed:', err.message);
        } else {
            console.log('Pipeline completed successfully');

            const size = fs.statSync('large-file-pipeline.txt.gz').size;
            console.log(`Compressed file: ${(size / 1024 / 1024).toFixed(2)} MB`);
        }
    }
);
```

```
node pipeline-demo.js
```

Output:

```
Pipeline completed successfully
Compressed file: 1.18 MB
```

What is happening? `pipeline()` takes any number of streams as arguments, followed by a callback. It pipes them together and handles cleanup: if any stream errors or closes prematurely, all streams in the pipeline are destroyed. This is the recommended approach for production code.

> **Warning:** Prefer `pipeline()` over manual `.pipe()` chains in production code. Manual piping does not clean up resources on error, which can cause file descriptor leaks and memory issues in long-running applications.

---

6. Transform Streams

A Transform stream sits between a Readable and a Writable stream, modifying data as it passes through. You have already seen one — `zlib.createGzip()`. Now let us build our own.

6.1 A Simple Transform — Uppercase Converter

Stage 1 **Basic Transform stream** `transform-uppercase.js`

```
const { Transform } = require('stream');

const toUpperCase = new Transform({
    transform(chunk, encoding, callback) {
        // chunk is a Buffer by default
        const upperChunk = chunk.toString().toUpperCase();
        callback(null, upperChunk);
    }
});

// Test it: pipe stdin through our transform to stdout
process.stdin.pipe(toUpperCase).pipe(process.stdout);
```

Run it and type some text (press `Ctrl` + `C` to exit):

```
node transform-uppercase.js
```

Output:

```
hello world
HELLO WORLD
streams are powerful
STREAMS ARE POWERFUL
```

What is happening? The `Transform` constructor accepts an options object with a `transform` method. This method receives each chunk, processes it, and calls `callback(error, transformedData)`. Passing `null` as the first argument means no error. The second argument is the transformed chunk that gets pushed downstream.

6.2 Transform with File I/O — Line Numbering

Stage 2 **A more practical transform**

Let us create a transform that adds line numbers to each line of a file:

transform-line-numbers.js

```
const fs = require('fs');
const { Transform, pipeline } = require('stream');

let lineNumber = 0;
let leftover = '';

const addLineNumbers = new Transform({
    transform(chunk, encoding, callback) {
        // Combine leftover from previous chunk with current chunk
        const text = leftover + chunk.toString();
        const lines = text.split('\n');

        // The last element may be incomplete — save it for the next chunk
        leftover = lines.pop();

        let output = '';
        for (const line of lines) {
            lineNumber++;
            output += `${String(lineNumber).padStart(6, ' ')} | ${line}\n`;
        }

        callback(null, output);
```

```
    },

    flush(callback) {
        // Handle any remaining data after the stream ends
        if (leftover) {
            lineNumber++;
            callback(null, `${String(lineNumber).padStart(6, ' ')} | ${leftover}\n`);
        } else {
            callback();
        }
    }
});

pipeline(
    fs.createReadStream('large-file.txt', { encoding: 'utf8' }),
    addLineNumbers,
    fs.createWriteStream('numbered-output.txt'),
    (err) => {
        if (err) {
            console.error('Pipeline failed:', err.message);
        } else {
            console.log(`Done. Added line numbers to ${lineNumber} lines.`);

            // Show first 5 lines of output
            const preview = fs.readFileSync('numbered-output.txt', 'utf8');
            const firstFive = preview.split('\n').slice(0, 5).join('\n');
            console.log('\nPreview:');
            console.log(firstFive);
        }
    }
);
```

```
node transform-line-numbers.js
```

Output:

```
Done. Added line numbers to 1000000 lines.

Preview:
     1 | Line 1: This is sample data for demonstrating streams in Node.js
     2 | Line 2: This is sample data for demonstrating streams in Node.js
     3 | Line 3: This is sample data for demonstrating streams in Node.js
     4 | Line 4: This is sample data for demonstrating streams in Node.js
     5 | Line 5: This is sample data for demonstrating streams in Node.js
```

What is happening? Two important patterns are demonstrated here:

Handling line boundaries across chunks: A chunk does not respect line boundaries — a chunk might end in the middle of a line. We handle this by saving any incomplete text in `leftover` and prepending it to the next chunk.

The `flush()` method: This is called once, after all data has been consumed but before the `'end'` event. It gives you a chance to push any remaining buffered data. Here, we use it to number the final line if it did not end with a newline character.

> **Note:** The pattern of handling leftover data across chunk boundaries is extremely common in stream programming. Any time you process line-based data (logs, CSV, configuration files), you will need this pattern.

6.3 Duplex Streams — A Conceptual Overview

A Duplex stream is both readable and writable, but unlike a Transform stream, the read and write sides are independent — data written to it does not automatically appear on the read side. The most common example is a TCP socket: you write data to send it to the remote host, and you read data that arrives from the remote host.

You will encounter Duplex streams when working with network sockets (covered in Section 6). For now, understand that a Transform stream is a *specialised* Duplex stream where the writable side feeds into the readable side through a transformation function.

---

7. Buffers

Throughout the examples above, you may have noticed references to "chunks" being Buffers by default. A **Buffer** is Node.js's way of handling raw binary data — sequences of bytes that exist outside of V8's normal string/object heap.

Why does this matter? JavaScript strings are UTF-16 encoded, which is efficient for text but cannot represent arbitrary binary data (images, compressed files, network packets). Buffers fill this gap.

7.1 Creating Buffers

buffer-creation.js

```
// From a string
const buf1 = Buffer.from('Hello, Node.js!');
console.log('From string:', buf1);
console.log('Length (bytes):', buf1.length);
console.log('As string:', buf1.toString());

console.log('---');

// From an array of byte values
const buf2 = Buffer.from([72, 101, 108, 108, 111]);
console.log('From bytes:', buf2);
console.log('As string:', buf2.toString());

console.log('---');

// Allocate a fixed-size buffer (filled with zeros)
const buf3 = Buffer.alloc(10);
console.log('Allocated:', buf3);

// Write data into it
buf3.write('Node');
console.log('After write:', buf3);
console.log('As string:', buf3.toString());
```

node buffer-creation.js

Output:

```
From string: <Buffer 48 65 6c 6c 6f 2c 20 4e 6f 64 65 2e 6a 73 21>
Length (bytes): 15
As string: Hello, Node.js!
---
From bytes: <Buffer 48 65 6c 6c 6f>
As string: Hello
---
Allocated: <Buffer 00 00 00 00 00 00 00 00 00 00>
After write: <Buffer 4e 6f 64 65 00 00 00 00 00 00>
As string: Node
```

What is happening?

`Buffer.from(string)` encodes a string into bytes using UTF-8 by default. Each byte is shown as a two-digit hexadecimal value — `48` is the ASCII code for "H", `65` for "e", and so on.

`Buffer.from(array)` creates a Buffer from raw byte values. The values `[72, 101, 108, 108, 111]` are the decimal equivalents of the same hex values, spelling "Hello".

`Buffer.alloc(size)` creates a zero-filled Buffer of a fixed size. You can then write into it with the `write()` method. The remaining bytes stay as `00`.

> **Warning:** Never use `Buffer.allocUnsafe()` unless you understand the security implications. It allocates memory without zeroing it out, meaning the buffer may contain old data from previous allocations. Use `Buffer.alloc()` for safety.

7.2 Encoding and Decoding

Buffers can convert between different text encodings. The most common encodings are `utf8`, `ascii`, `base64`, and `hex`.

buffer-encoding.js

```
const original = 'Streams and Buffers';

// Encode to different formats
const buf = Buffer.from(original, 'utf8');

console.log('UTF-8 (default):', buf.toString('utf8'));
console.log('Base64:         ', buf.toString('base64'));
console.log('Hex:            ', buf.toString('hex'));

console.log('---');

// Decode from Base64
const base64String = 'Tm9kZS5qcyBpcyBhc3luY2hyb25vdXM=';
const decoded = Buffer.from(base64String, 'base64');
console.log('Decoded from Base64:', decoded.toString('utf8'));

console.log('---');

// Practical: encoding a string for safe URL transmission
const data = 'user=admin&role=instructor';
const encoded = Buffer.from(data).toString('base64');
```

```
console.log('Original:    ', data);
console.log('Base64:      ', encoded);
console.log('Decoded back:', Buffer.from(encoded, 'base64').toString('utf8'));
```

`node buffer-encoding.js`

Output:

```
UTF-8 (default): Streams and Buffers
Base64:          U3RyZWFtcyBhbmQgQnVmZmVycw==
Hex:             53747265616d7320616e642042756666657273
---
Decoded from Base64: Node.js is asynchronous
---
Original:     user=admin&role=instructor
Base64:       dXNlcj1hZG1pbiZyb2xlPWluc3RydWN0b3I=
Decoded back: user=admin&role=instructor
```

What is happening? The `toString(encoding)` method converts a Buffer's bytes into a string using the specified encoding. Base64 encoding is commonly used to transmit binary data as text — you will encounter it with authentication tokens, file uploads, and API payloads.

7.3 Buffers in Streams

When a stream does not have an `encoding` set, chunks arrive as Buffers. This is important to understand when working with non-text data:

`buffer-in-streams.js`

```
const fs = require('fs');

// Without encoding — chunks are Buffers
const raw = fs.createReadStream('large-file.txt', {
    highWaterMark: 64
});

raw.once('data', (chunk) => {
    console.log('Type:', typeof chunk, '|', chunk.constructor.name);
    console.log('Raw:', chunk);
    console.log('As string:', chunk.toString('utf8'));
});

// With encoding — chunks are strings
```

```
const text = fs.createReadStream('large-file.txt', {
    encoding: 'utf8',
    highWaterMark: 64
});

text.once('data', (chunk) => {
    console.log('\nType:', typeof chunk);
    console.log('Chunk:', chunk);
});
```

`node buffer-in-streams.js`

Output:

```
Type: object | Buffer
Raw: <Buffer 4c 69 6e 65 20 31 3a 20 54 68 69 73 20 69 73 20 73 61 6d 70 6c 65 20 64 61 74 61 20 66 6f 72 20 64
65 6d 6f 6e 73 74 72 61 74 69 6e 67 20 73 74 72 ... 14 more bytes>
As string: Line 1: This is sample data for demonstrating streams in Node.js


Type: string
Chunk: Line 1: This is sample data for demonstrating streams
```

When processing text files, setting `encoding: 'utf8'` is convenient. When processing binary files (images, compressed data), leave it unset to work with raw Buffers.

7.4 Concatenating Buffers

A common pattern is collecting all chunks from a stream and combining them into a single Buffer:

`buffer-concat.js`

```
const fs = require('fs');

const chunks = [];

const readStream = fs.createReadStream('output.txt');

readStream.on('data', (chunk) => {
    chunks.push(chunk);
});

readStream.on('end', () => {
    const fullBuffer = Buffer.concat(chunks);
```

```
    console.log('Total bytes:', fullBuffer.length);
    console.log('Content:', fullBuffer.toString('utf8'));
});
```

```
node buffer-concat.js
```

Output:

```
Total bytes: 119
Content: First line of output
Second line of output
Third line of output
Final line — stream closed after this
```

`Buffer.concat()` takes an array of Buffers and joins them into a single contiguous Buffer. Use this pattern when you need the complete data but still want the memory efficiency of streaming during the read phase.

> **Note:** Be mindful that `Buffer.concat()` on a very large file defeats the purpose of streaming, since you end up with all the data in memory anyway. Use it only when you genuinely need the complete data — for example, when parsing a small-to-medium JSON file received as a stream.

Summary

Concept	Key Point
Streams	Process data in chunks rather than loading everything into memory; essential for large files and network I/O
Readable Stream	Source of data; emits `'data'`, `'end'`, and `'error'` events; can be paused and resumed
Writable Stream	Destination for data; `write()` returns `false` when buffer is full; `'drain'` signals readiness

Backpressure	Mechanism for a slow consumer to signal a fast producer to pause; handled automatically by `pipe()`
pipe()	Connects readable → writable with automatic backpressure; chainable through Transform/ Duplex streams
pipeline()	Preferred over `pipe()` — properly destroys all streams on error; takes a callback for completion
Transform Stream	Modifies data between read and write; implement `transform()` and optionally `flush()`
Duplex Stream	Independently readable and writable (e.g., TCP sockets); Transform is a specialised subtype
Buffer	Represents raw binary data; created via `Buffer.from()` or `Buffer.alloc()`
Encoding	Convert between formats with `toString(encoding)` : utf8, base64, hex, ascii

Interview Questions

1. What problem do streams solve in Node.js, and when would you use them over `readFile()`?
Expected answer: Streams process data in chunks, keeping memory usage constant regardless of data size. Use streams when working with large files, network data, or any I/O where you do not need the entire dataset in memory at once. `readFile()` loads everything into memory, which fails or degrades performance with large files.

2. Name the four types of streams in Node.js and give an example of each.
Expected answer: Readable (e.g., `fs.createReadStream()`), Writable (e.g., `fs.createWriteStream()`), Duplex (e.g., TCP socket — independent read/write channels), and Transform (e.g., `zlib.createGzip()` — modifies data passing through).

3. What is backpressure and how does Node.js handle it?
Expected answer: Backpressure occurs when a producer generates data faster than a consumer can process it. In Node.js, `writable.write()` returns `false` when the internal buffer exceeds `highWaterMark`. The producer should stop writing and wait for the `'drain'` event. `pipe()` handles this automatically.

4. Why should you prefer `pipeline()` over `.pipe()` in production code?
Expected answer: `pipe()` does not automatically clean up streams if an error occurs mid-chain, leading to resource leaks (open file descriptors, memory). `pipeline()` properly destroys all streams on error and provides a single callback for error handling and completion.

5. How do you handle line boundaries when processing text with streams?
Expected answer: Chunks do not align with line boundaries. You need to maintain a "leftover" variable: split each chunk by newline, save the last element (which may be incomplete) for the next chunk, and process the complete lines. Use the `flush()` method to handle any remaining data after the stream ends.

6. What is a Buffer and why does Node.js need it?
Expected answer: A Buffer is a fixed-size allocation of raw binary data outside V8's heap. JavaScript strings are UTF-16 encoded and cannot represent arbitrary binary data. Buffers allow Node.js to work with binary data from files, network packets, and other I/O operations efficiently.

7. Explain the difference between `Buffer.alloc()` and `Buffer.allocUnsafe()`.
Expected answer: `Buffer.alloc(size)` creates a zero-filled buffer, which is safe but slightly slower. `Buffer.allocUnsafe(size)` allocates memory without initialising it — the buffer may contain old data from previous memory allocations, which is a security risk. Use `alloc()` unless you have a performance-critical reason and will immediately overwrite the contents.

8. If you pipe a Readable stream through a Transform and into a Writable, and the Writable throws an error, what happens to the other streams?
Expected answer: With `.pipe()`, the other streams are *not* automatically destroyed — they remain open, potentially leaking resources. With `pipeline()`, all streams in the chain are properly destroyed when any stream errors.

Practical Assignment — Log File Processor

Objective: Build a stream-based log file processor that reads a raw CSV log file, transforms each entry, and writes a formatted report — all using streams to handle files of any size.

Step 1: Create a project folder and navigate into it:

```
mkdir log-processor && cd log-processor
```

Step 2: Create a script to generate a sample CSV log file:

`generate-logs.js`

```
const fs = require('fs');

const levels = ['INFO', 'WARN', 'ERROR', 'DEBUG'];
const messages = [
    'User logged in',
    'Database query executed',
    'Cache miss detected',
    'File upload completed',
    'API request timeout',
    'Memory usage above threshold',
    'Session expired',
    'Invalid input received',
    'Connection pool exhausted',
    'Scheduled task completed'
];

const writeStream = fs.createWriteStream('server.log.csv');

// Write CSV header
```

```
writeStream.write('timestamp,level,message,responseTime\n');

const totalEntries = 50000;

for (let i = 0; i < totalEntries; i++) {
    const date = new Date(2025, 0, 1, Math.floor(Math.random() * 24),
        Math.floor(Math.random() * 60), Math.floor(Math.random() * 60));
    const timestamp = date.toISOString();
    const level = levels[Math.floor(Math.random() * levels.length)];
    const message = messages[Math.floor(Math.random() * messages.length)];
    const responseTime = Math.floor(Math.random() * 2000) + 50;

    writeStream.write(`${timestamp},${level},${message},${responseTime}\n`);
}

writeStream.end(() => {
    const size = fs.statSync('server.log.csv').size;
    console.log(`Generated ${totalEntries} log entries (${(size / 1024).toFixed(1)}
KB)`);
});
```

Step 3: Run the generator:

```
node generate-logs.js
```

Step 4: Create the log processor using streams and a custom Transform:

process-logs.js

```
const fs = require('fs');
const { Transform, pipeline } = require('stream');

// --- Statistics collector ---
const stats = {
    total: 0,
    byLevel: { INFO: 0, WARN: 0, ERROR: 0, DEBUG: 0 },
    totalResponseTime: 0,
    slowRequests: 0    // responseTime > 1500ms
};

// --- Transform: Parse CSV and filter/format ---
let leftover = '';
let isHeader = true;

const processLogs = new Transform({
```

```
    transform(chunk, encoding, callback) {
        const text = leftover + chunk.toString();
        const lines = text.split('\n');
        leftover = lines.pop(); // Save incomplete line

        let output = '';

        for (const line of lines) {
            // Skip the CSV header row
            if (isHeader) {
                isHeader = false;
                continue;
            }

            const parts = line.split(',');
            if (parts.length < 4) continue; // Skip malformed lines

            const [timestamp, level, message, responseTimeStr] = parts;
            const responseTime = parseInt(responseTimeStr, 10);

            // Update statistics
            stats.total++;
            if (stats.byLevel[level] !== undefined) {
                stats.byLevel[level]++;
            }
            stats.totalResponseTime += responseTime;
            if (responseTime > 1500) {
                stats.slowRequests++;
            }

            // Only include WARN and ERROR entries in the output
            if (level === 'WARN' || level === 'ERROR') {
                const date = new Date(timestamp);
                const formatted = date.toLocaleString('en-GB', {
                    day: '2-digit', month: 'short', year: 'numeric',
                    hour: '2-digit', minute: '2-digit', second: '2-digit'
                });

                output += `[${formatted}] [${level.padEnd(5)}] ${message} ($
{responseTime}ms)\n`;
            }
        }

        callback(null, output);
    },

    flush(callback) {
```

```
        // Process any remaining data
        if (leftover && leftover.trim()) {
            const parts = leftover.split(',');
            if (parts.length >= 4) {
                stats.total++;
                const level = parts[1];
                if (stats.byLevel[level] !== undefined) {
                    stats.byLevel[level]++;
                }
            }
        }
        callback();
    }
});

// --- Run the pipeline ---
pipeline(
    fs.createReadStream('server.log.csv', { encoding: 'utf8' }),
    processLogs,
    fs.createWriteStream('filtered-warnings-errors.log'),
    (err) => {
        if (err) {
            console.error('Processing failed:', err.message);
            return;
        }

        // Print summary statistics
        const avgResponse = (stats.totalResponseTime / stats.total).toFixed(1);

        console.log('=== Log Processing Complete ===\n');
        console.log(`Total entries processed: ${stats.total}`);
        console.log(`  INFO:  ${stats.byLevel.INFO}`);
        console.log(`  WARN:  ${stats.byLevel.WARN}`);
        console.log(`  ERROR: ${stats.byLevel.ERROR}`);
        console.log(`  DEBUG: ${stats.byLevel.DEBUG}`);
        console.log(`\nAverage response time: ${avgResponse}ms`);
        console.log(`Slow requests (>1500ms): ${stats.slowRequests}`);
        console.log(`\nFiltered output written to: filtered-warnings-errors.log`);
    }
);
```

Step 5: Run the processor:

```
node process-logs.js
```

Expected output:

```
Output:
=== Log Processing Complete ===

Total entries processed: 50000
  INFO:  12534
  WARN:  12488
  ERROR: 12491
  DEBUG: 12487

Average response time: 1074.3ms
Slow requests (>1500ms): 12456

Filtered output written to: filtered-warnings-errors.log
```

(Your exact numbers will vary due to random generation.)

Step 6: Verify the output file (macOS/Linux: `head -5 filtered-warnings-errors.log`, Windows: `powershell -Command "Get-Content filtered-warnings-errors.log -TotalCount 5"`):

```
Output:
[01 Jan 2025, 03:42:18] [WARN ] Cache miss detected (1823ms)
[01 Jan 2025, 14:07:51] [ERROR] API request timeout (432ms)
[01 Jan 2025, 22:15:03] [WARN ] Memory usage above threshold (1901ms)
[01 Jan 2025, 09:33:27] [ERROR] Connection pool exhausted (67ms)
[01 Jan 2025, 18:56:44] [WARN ] Invalid input received (1102ms)
```

```
log-processor/
├── generate-logs.js
├── process-logs.js
├── server.log.csv
└── filtered-warnings-errors.log
```

Bonus Challenge

Extend the processor to also compress the output file using `zlib.createGzip()`. The pipeline should be: read CSV → transform → gzip → write `.gz` file. Print both the uncompressed and compressed file sizes in the summary to see the compression ratio.